

Agentic Coding

Drei Module aus dem Workshop am 30. April 2026

1. **AGENTS.md** Theorie, Muster und zwei Hands-On-Übungen
2. **Deep Dive** Was zwischen Coding Agent und LLM passiert
3. **Echte Captures** Vier Tools, eine Aufgabe — gemessen am Proxy

Wie schreibt man eine, die wirklich hilft?

Erkenntnisse aus einer empirischen Studie + zwei Hands-On-Übungen

Das „Markdown-Museum“ — warum AGENTS.md?

Jedes Tool brachte seinen eigenen Standard — Entwickler pflegten dieselben Regeln mehrfach.

Tool	Proprietäre Datei
Claude Code	CLAUDE.md
Gemini CLI	GEMINI.md
Cursor IDE	.cursorrules / .cursor/rules/
GitHub Copilot	.github/copilot-instructions.md
Windsurf	.windsurf/rules
Google Jules	JULES.md

Konvergenz

AGENTS.md

Ein Standard, ein Ort.

> 20 000 OSS-Repos

Unterstützt von OpenAI/Codex, Cursor, Amp, Factory, Jules u.a.

Pragmatisch: Wer Claude Code nutzt, kann CLAUDE.md → AGENTS.md symlinken. Eine Quelle, alle Tools.

Kernbefund der Studie

Augment Code, AuggieBench-Eval auf echten PRs aus dem Monorepo

Eine gute AGENTS.md = Modell-Upgrade

Eine schlechte AGENTS.md = schlechter als gar keine.

+25 %

Best-Practice-Score auf einem Bugfix dank Decision-Table

−30 %

Vollständigkeit auf einem Feature-Task — gleiche Datei, anderer Kontext

Was funktioniert — 6 Muster

Empirisch belegt durch A/B-Vergleich auf realen Tasks

1

Progressive Disclosure

100–150 Zeilen + Reference-Files
statt einem Mega-Dokument

2

Prozedurale Workflows

Nummerierte Schritte für
wiederkehrende Aufgaben

3

Decision Tables

Mehrdeutigkeit auflösen,
bevor der Agent Code schreibt

4

Echte Code-Beispiele

3–10 Zeilen aus Produktion,
nicht erfundene Snippets

5

Don't + Do paaren

Zu jedem Verbot eine
konkrete Alternative

6

Modular pro Submodul

Module-AGENTS.md schlägt
Monster-Repo-Root-Datei

Detail-Folien zu Mustern 2, 3, 5 folgen

Decision Tables — Entscheidungen vor dem Schreiben

Wenn 2–3 Wege existieren: gib dem Agent ein Schema, keine Prosa.

Frage → React Query → Zustand

Server ist die einzige Datenquelle?



Mehrere Code-Pfade mutieren den State?



Optimistische Updates + lokaler State?



Beobachtetes Ergebnis

+25 %

auf best_practices — die Tabelle löst die Wahl, bevor der Agent eine Zeile schreibt.

Prozedurale Workflows — nummeriert, schrittweise

Der stärkste gemessene Effekt: vom „kann nicht“ zum „beim ersten Versuch fertig“.

Beispiel: „Neue Integration ausrollen“

1. Konfiguration in ``integrations/`` registrieren
2. Schema in ``schemas/<name>.ts`` hinzufügen
3. Handler in ``handlers/`` mit Retry-Wrapper
4. Eintrag in ``registry.ts`` (alphabetisch)
5. Test in ``__tests__/integrations/``
6. Eintrag in ``docs/integrations.md``

Gemessener Effekt

40 % → 10 %

Anteil der PRs mit fehlenden Wiring-Files

+25 %

Korrektheit (correctness)

+20 %

Vollständigkeit (completeness)

Zu jedem „Don't" gehört ein „Do"

Reine Verbote machen den Agent vorsichtig und exploratorisch — die falsche Reaktion.

Nur „Don't"

× Don't instantiate HTTP clients directly.

Agent wird vorsichtig, exploriert mehr, schreibt weniger. Mit 15+ Don'ts kippt das Verhalten messbar.

„Don't" + „Do"

× Don't instantiate HTTP clients directly.

✓ Use the shared `apiClient` from `lib/http` with the `retry` middleware.

Agent weiß, was zu tun ist, und macht weiter. Kein Suchverhalten.

Anti-Pattern: Die Overexploration-Falle

Häufigste Fehlerart in der Studie — im Kern: Context Rot.

Zu viel Architektur-Übersicht

Aufgabe: 2-Zeilen-Config-Change.

Agent liest 12 Doku-Files, lädt ~80 K Tokens irrelevanten Kontext, verwirrt sich über Service-Ownership.

completeness: -25 %

30+ „Don't“-Regeln

Aufgabe: einfacher CRUD-Endpoint.

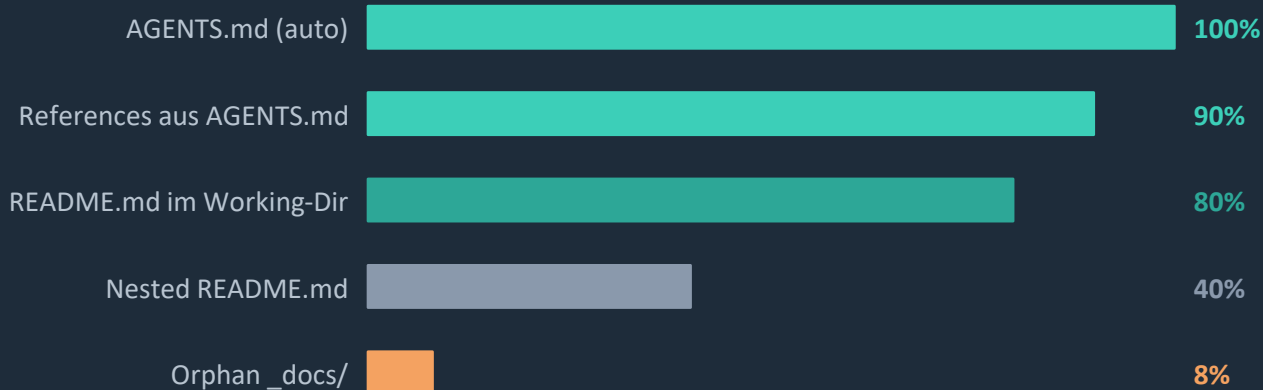
Agent prüft jeden Warning auf Relevanz, exploriert Migration-Skripte und Auth-Middleware — alles unnötig.

PR doppelt so lang, 20 % weniger vollständig

Fix: Architektur knapp & isoliert halten. Kern-Gotchas in der Hauptdatei, Rest in Reference-Files.

Wo schaut der Agent wirklich hin?

Discovery-Raten über hunderte Sessions — AGENTS.md ist der einzig zuverlässige Einstieg.



Praktische Folge

Soll etwas gesehen werden: in AGENTS.md schreiben oder direkt von dort referenzieren.

Orphan-Docs in _docs/ sind quasi unsichtbar.

Cheatsheet: Was willst du verbessern?

Verschiedene Muster bewegen verschiedene Metriken. Ziel zuerst, Muster danach.

Wenn du verbessern willst ...	... nimm dieses Muster
Wiederverwendung von Code	Wenige relevante Beispiele aus Prod-Code
Konsistenz mit Codebase-Konventionen	Decision Tables für Komponenten/Libs
Korrektes Wiring komplexer Features	Prozeduraler nummerierter Workflow
Umgang mit Gotchas	„Don't" gepaart mit „Do"
Context Rot vermeiden	Progressive Disclosure via Reference-Files
Context Rot vermeiden	Nur Inhalte mit Bezug zum umgebenden Code

ÜBUNG 1

20 Min

A/B/C-Test mit dem API-Proxy

Drei Runs, gleiche Aufgabe — sichtbar gemacht durch den XaresAICoder-Proxy.

Run A

Ohne AGENTS.md

Agent startet blind. Baseline für Token-Verbrauch und gelesene Dateien.

Run B

„Schlechte“ AGENTS.md

Lange Architektur-Übersicht + 30 Don'ts ohne Dos. Beobachte Overexploration.

Run C

„Gute“ AGENTS.md

~100 Zeilen: 1 Decision-Table, 1 Workflow, 5 Don't/Do-Paare. Vergleiche.

Im Proxy beobachten

Tokens In/Out · Anzahl Tool-Calls (read/grep) · Welche Files wurden gelesen? · Korrektheit & Vollständigkeit des Ergebnisses

Vorbereitete AGENTS.md-Varianten liegen im Workshop-Repo

ÜBUNG 2

25 Min

Schreib deine eigene AGENTS.md

Eigenes Projekt oder Workshop-Beispiel. Maximal 100 Zeilen. Nichts Generisches.

Pflicht-Bestandteile

- 1 Decision-Table für eine echte Wahl im Projekt
- 1 nummerierter Workflow für einen wiederkehrenden Task
- 3–5 Don't/Do-Paare statt einer Verbots-Liste
- 2–3 Code-Snippets (3–10 Zeilen) aus dem realen Code

Was ihr weglasst

- ✗ Lange Architektur-Erklärungen mit „warum“
- ✗ Mehr als 10 Don'ts ohne passende Dos
- ✗ Allgemeine Coding-Best-Practices ohne Projekt-Bezug
- ✗ Verweise auf Doks, die niemand pflegt

Was wir im Workshop wirklich gesehen haben

A/B/C-Test mit aktuellen Modellen (OpenCode auf Qwen3.6, Gemini CLI) — der Spread fiel kleiner aus als die Studie nahelegt.

Beobachtung

Selbst Run B („schlechte“ AGENTS.md) führte nur teilweise zu schlechterem Code. Generischer Architektur-Boilerplate wird vom Modell weitgehend als Rauschen behandelt — die Modelle sind besser geworden im Filtern.

Was greift

Projekt-spezifisches Wissen, das das Modell nicht raten kann. Z.B. XaresAICoder: „bind 0.0.0.0 statt localhost“. Oder echte Coding-Conventions: Klammer-Stil, TDD, Fehlerbehandlungs-Pattern.

Was kaum noch wirkt

Generische „Architektur-Übersichten“, Compliance-Listen, Mega-Don't-Listen ohne Projekt-Bezug. Vor einem Jahr hat das spürbar gestört — heute filtert es das Modell weitgehend selbst raus.

Konsequenz für die Übung: Anti-Pattern-Variante muss schärfer akzentuiert werden, damit der A/B/C-Effekt 2026 noch deutlich sichtbar wird.

- 01 AGENTS.md ist nicht „mehr Doku“ — sie ist ein Steuerungsinstrument für Token & Aufmerksamkeit.
- 02 Decision-Tables und Workflows sind die effizientesten Muster pro Zeile.
- 03 Jedes „Don't“ braucht ein „Do“. Sonst exploriert der Agent statt zu liefern.
- 04 Modul-AGENTS.md > Repo-Root-Mega-Datei. 100–150 Zeilen + Reference-Files.
- 05 Was nicht in AGENTS.md steht oder von dort verlinkt ist, sieht der Agent nur unzuverlässig.

Quelle

Slava Zhenylenko, Augment Code (Apr 2026): „A good AGENTS.md is a model upgrade. A bad one is worse than no docs at all.“ · augmentcode.com/blog/how-to-write-good-agents-dot-md-files

Was passiert unter der Haube?

API-Proxy-Demo · Token-Ökonomie · Tool-Vergleich



Die zentrale Erkenntnis

Das LLM weiß nichts von eurem Projekt.

Jeder API-Call startet bei null. Alles, was wie Gedächtnis, Projektverständnis und Kontext aussieht, baut der Agent vor jedem Call neu zusammen.

Stateless

Kein Hidden State auf der LLM-Seite

Tool-Leistung

Was wie „Verstehen“ wirkt, ist Vorarbeit des Tools

Steuerbar

Wer den Request versteht, kann ihn formen

Was steckt wirklich in einem Request?

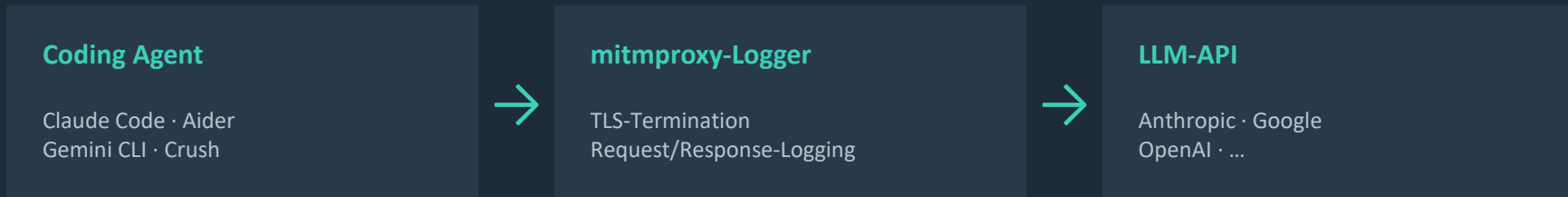
Eine einzige „simple Frage“ ist im Hintergrund ein Stack aus mehreren Schichten.

System-Prompt	Tool-Persona, Regeln, oft 5.000+ Zeichen
Tool-Definitionen	JSON-Schemas für jedes verfügbare Werkzeug
Conversation History	Alle bisherigen Turns dieser Session
Datei-/Projekt-Kontext	Auszüge, Repo-Map, AGENTS.md, Tool-Outputs
User-Prompt	Der eine Satz, den der Mensch tippt

Pointe: Der User-Prompt ist meist die kleinste Schicht im Stack.

Live-Demo: Der API-Proxy in XaresAICoder

Sichtbar machen, was sonst unsichtbar bleibt — ohne das Tool-Verhalten zu verändern.



Was wir gleich sehen:

Vollständige Requests

System-Prompts, Tool-Definitionen, History

Token-Usage pro Call

Input · Output · Cached

Streaming-Responses

SSE-Events live mitgeschnitten

Differenzen zwischen Tools

Was packt wer in den Payload?

Drei Tools, drei Architekturen

Gleiche Aufgabe — völlig unterschiedliche Strategien, den Kontext zusammenzubauen.

Aider

sparsam

- Tree-Sitter parst die ganze Codebasis
- Komprimiert sie zu einer Repository-Map
- Nur relevante Symbole landen im Prompt

≈ 98%

Token-Ersparnis ggü. Voll-Files

Claude Code

autonom

- Orchestriert Sub-Agenten
- Erkundet das Dateisystem aktiv
- Mächtig, aber Token-hungrig

Sub-Agenten

Eigene Calls für jede Exploration

Gemini CLI

redundant

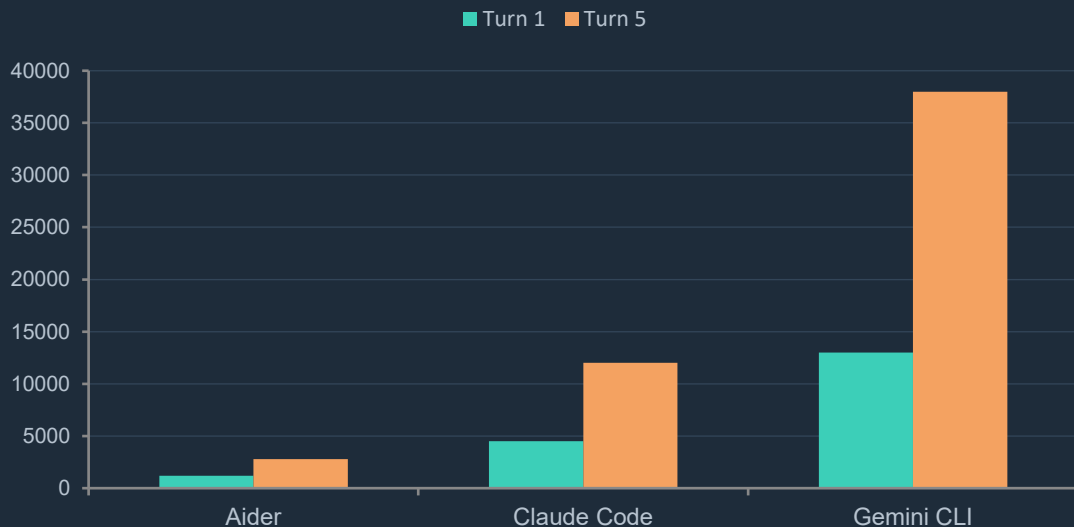
- Rebuilt den Payload bei jedem Turn
- System-Prompt + Tool-Defs jedes Mal
- Verstecktes Routing wählt das Modell

13.000+

Tokens für 2 simple Fragen

Token-Ökonomie sichtbar gemacht

Was passiert mit dem Token-Verbrauch über mehrere Turns?



Input ≠ Output

Eingabe-Token dominieren — 10–100× über der Antwort.

Linear, nicht konstant

Jeder neue Turn bringt mehr History und schiebt die Kurve hoch.

Tool-Wahl entscheidet

Die Architektur des Tools bestimmt die Steigung.

Prompt Caching

Der Trick gegen explodierende Kosten in langen Sessions.

Ohne Caching

- System-Prompt wird bei jedem Turn neu prozessiert
- Tool-Definitionen ebenso — jedes Mal voller Preis
- In langen Sessions: Kosten steigen linear mit Turns

1×

Voller Input-Token-Preis

Mit Caching

- Stabile Präfixe (System, Tools) werden gecacht
- Cache-Hit: ein Bruchteil des normalen Preises
- Tool muss Cache-Marker explizit setzen

≈ 0,1×

Cache-Hit (Anthropic-Tarif)

Mit dieser Brille auf den Workshop-Tag

Kontext bewusst formen

AGENTS.md, CONVENTIONS.md & Co. wirken, weil sie im Request landen.

Tool zur Aufgabe wählen

Sparsam, autonom oder redundant — jede Architektur hat ihren Sweet Spot.

Kosten kontrollieren

Caching nutzen, Sessions begrenzen, History gezielt zurücksetzen.

Sicherheitsbrille auf

Was automatisch im Payload landet, kann auch Einfallstor sein.

Echte Zahlen — vier Tools, eine Aufgabe

Express-Codebase · 5 Turns (1 Erkundung · 3 Folgefragen · 1 Edit) · alles im Proxy mitgeschnitten

aider

anthropic, opencode · big-pickle,
claude-haiku-4-5-20251001, cla

86.398

tokens input total

12 Calls · 82 output

opencode

opencode · big-pickle, gpt-5-nano

131.073

tokens input total

11 Calls · 1.973 output

gemini

google · gemini-3-flash-preview

177.024

tokens input total

11 Calls · 3.486 output

claude

anthropic · claude-haiku-4-5-20251001,
claude-opus-4-7

263.084

tokens input total

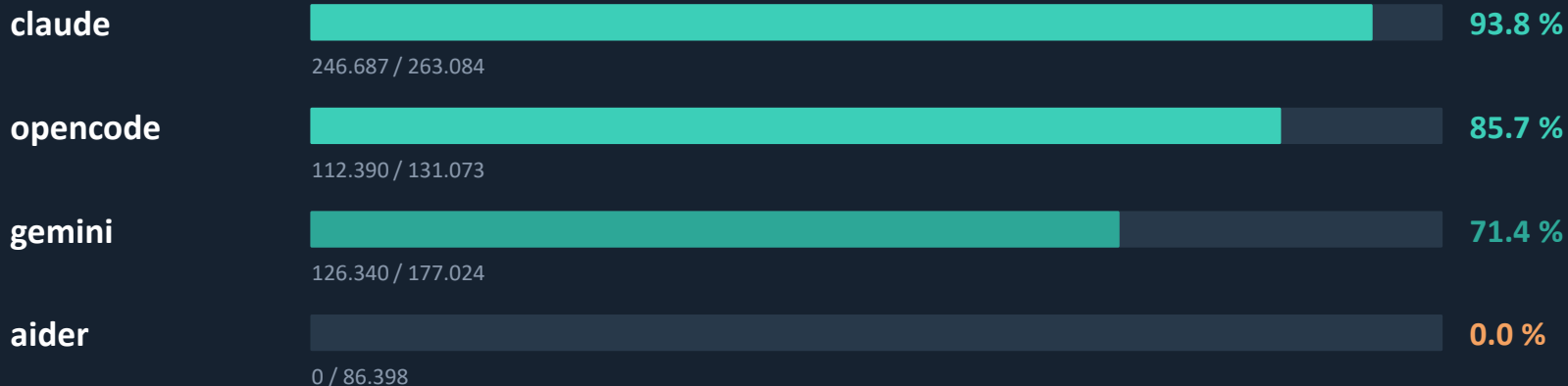
12 Calls · 338 output

Verhältnis 1 : 3

Aider zieht ~86.398 Input-Tokens — Claude Code ~263.084. Drei Mal mehr für dieselbe Aufgabe.

Cache-Hit-Rate ist real

cache_read_input_tokens aus den echten Captures — bestätigt Slide „Prompt Caching“



Anthropic-Tarif: $\text{cache_read} \approx 0,1 \times$ normalem Input — Claude Code würde ohne Caching $\sim 17 \times$ kosten · Auch Gemini 3 cached (71 %). Aider macht keins — System-Prompt klein genug.

Verstecktes Routing — was OpenCode wirklich tut

Der Proxy zeigt: hinter „OpenCode AI“ steckt Qwen 3.6 Plus (Alibaba) — verpackt im Anthropic-Format unter dem Codenamen *big-pickle*.

Nutzerbefehl im Workspace

```
$ opencode
```

Free tier ausgewählt — kein Konto, keine Karte.
Tool meldet sich als "connected to opencode AI".



Was der Proxy mitschneidet

```
POST opencode.ai/zen/v1/  
"model": "big-pickle"
```

Response-Body verrät: "model": "qwen3.6-plus"



Tatsächliches Modell

= Qwen 3.6 Plus

Alibaba Qwen — Anthropic-kompatibler Wrapper, damit westliche Tools ohne Anpassung gehen.

Was lernt man daraus?

- Ohne Proxy weißt du nicht, welches Modell deine Daten wirklich verarbeitet.
- Anthropic-Format ≠ Anthropic-Modell.
- Free tier ohne Anmeldung = günstige Inferenz im Hintergrund.

Vier Take-aways aus dem Live-Mitschnitt

Was dieser Workshop konkret gezeigt hat — mit Zahlen statt Folklore.

3× Spread bei Token-Verbrauch

aider 86.398 → claude 263.084 für dieselbe Aufgabe. Wahl der Architektur wirkt direkt auf die Rechnung.

Caching ist real und billig

Claude Code: 94 % Cache-Hit. Ohne Caching wäre dieselbe Session ~17× teurer. Auch Gemini 3 cached (71 %).

Aider ist sparsam — und ehrlich

Tree-sitter-Map + Edit-Block-Format: 86k Tokens, kein Caching nötig, Output-Diff statt vollem File.

Modell hinter dem CLI nicht immer transparent

OpenCode free tier: Header sagt „big-pickle“, Body verrät „qwen3.6-plus“ (Alibaba Qwen).

Vorschau auf mehr Details – JFS 2026

Im Workshop angerissen — beim JFS 2026 in voller Tiefe

Dein Coding Agent belügt dich nicht – er vergisst dich

Den Proxy und die zentrale These habt ihr im Workshop gesehen. Beim JFS wird daraus ein voller Vier-Tools-Vergleich — mit Codex CLI, vollständigen Captures und den Praxis-Folgen für Kontext, Kosten und Sicherheit.

Claude Code, Aider, Codex, Gemini CLI - sie wirken, als würden sie dein Projekt „verstehen“. Doch das LLM dahinter ist zustandslos: Jeder API-Call startet bei null. Was wie Gedächtnis aussieht, ist Leistung des Tools - nicht des Modells. Per API-Proxy wird sichtbar, was die vier Tools wirklich an das LLM schicken - und wie unterschiedlich sie es tun: Aiders Tree-Sitter-Repository-Map gegen Claude Codes Sub-Agenten gegen Codex' Responses API gegen Gemini CLIs verstecktes Routing zwischen Modellen. Eine Schicht tiefer als übliche Tool-Vergleiche: vom Anwenden zum Verstehen.

➤ Donnerstag, 9. Juli 2026 · 9:50 – 10:35 · Java Forum Stuttgart